Cronograma do Projeto GameLang

Disciplina: Projeto e Implementação de Compiladores — 2026/2

Objetivo do projeto

Desenvolver um compilador para a GameLang, uma linguagem específica para regras básicas de jogos. A primeira versão contempla entidades, propriedades numéricas, colisões, dano, cura, movimento, repetição e saída textual, com tradução para Python executável.

O cronograma segue a progressão prevista no plano de ensino: especificação da linguagem, análise léxica, parsing, análise semântica, representação intermediária, runtime, geração de código, otimização, testes, reprodutibilidade, artigo e apresentação final.

Cronograma detalhado

Data	Conteúdo do plano	Atividades GameLang	Entregáveis / marco
01/09	GLC, parsing descendente, LL(1), FIRST/FOLLOW	Revisar a especificação; formalizar a gramática EBNF; eliminar ambiguidades e recursões à esquerda; criar exemplos válidos e inválidos.	Gramática EBNF, tokens preliminares e testes iniciais.
08/09	Parsing ascendente, LR/SLR/LALR e conflitos	Definir e justificar a estratégia de parsing; implementar parser; criar AST; validar precedência e associatividade.	Parser inicial, AST e decisão arquitetural.
15/09	Recuperação de erros e qualidade dos diagnósticos	Implementar lexer: palavras-chave, identificadores, números, operadores, strings e comentários; registrar linha/coluna e erros léxicos.	Lexer funcional e catálogo de erros.
22/09	Tabela de símbolos, escopos e resolução de nomes	Criar tabela de símbolos; validar entidades, propriedades qualificadas como player.health, duplicidades e nomes inexistentes.	Analisador de nomes e regras de escopo.
29/09	Inferência de shapes, tipos e broadcasting	Definir tipos: inteiro, real, booleano, texto, entidade e posição; validar compatibilidade de operações e condições.	Sistema de tipos e testes semânticos.
06/10	IA #1: geração de código e avaliação de correção	Usar IA apenas como apoio controlado; registrar prompts; revisar código; ampliar testes de corretude e entradas inválidas.	Registro de IA e suíte de testes ampliada.
13/10	THIR, SSA e reescritas algébricas	Definir IR; converter AST para IR; criar temporários; imprimir IR; aplicar simplificações como x + 0 e x * 1.	Especificação da IR e conversor AST → IR.
20/10	Run-time, buffers, layout e interpretador	Implementar estado do jogo, eventos, colisão, dano, cura, movimento, repetição e saída; executar a IR em interpretador de referência.	Runtime e interpretador funcional.
27/10	Serialização de IR, PyO3 e arena em Rust	Serializar IR em JSON ou formato textual; modularizar o projeto; avaliar Rust/PyO3 como extensão opcional.	IR serializável, módulos e guia de instalação.
03/11	MLIR, TVM, XLA, Triton e separação algoritmo/escalonamento	Implementar backend GameLang → Python; comparar execução interpretada e código Python gerado; documentar decisões tecnológicas.	Gerador Python e testes de equivalência.
10/11	CFG, DCE, CSE, fusão e tiling	Criar CFG quando aplicável; implementar propagação de constantes, código morto, expressões comuns e redução de redundâncias.	Módulo de otimização e relatório de desempenho.
17/11	IA #3: autotuning, síntese e verificação	Criar entradas adversariais; verificar equivalência entre IR original e otimizada; registrar falhas e correções.	Relatório de testes adversariais; versão candidata.
18/11	Prova Interdisciplinar	Congelar funcionalidades principais; executar testes completos; corrigir erros críticos; preparar demonstração curta.	**M1:** pipeline completo funcional.
24/11	Metodologia e escrita científica; Projeto Integrador	Consolidar documentação e experimentos; estruturar artigo: problema, linguagem, arquitetura, metodologia, resultados e limitações.	**M2:** projeto integrador e rascunho do artigo.
01/12	Revisão por pares, rigor e integridade acadêmica; Quiz	Revisão cruzada do artigo e do código; conferir autoria, citações, reprodutibilidade e resultados; preparar slides.	**M3:** artigo revisado e apresentação preliminar.
08/12	Integração final e operador surpresa	Integrar lexer, parser, semântica, IR, otimização e backend; testar em ambiente limpo; incluir pequena extensão opcional.	**M4:** versão final e manual de uso.
12/12	Colaboração em grupo e Artigo Científico	Entregar código, gramática, testes, exemplos, instruções de execução, artigo, relatório de limitações e registro de IA.	**M5:** entrega final.
15/12	Apresentação do artigo	Demonstrar código GameLang, AST/IR, código Python gerado, tratamento de erro e uma otimização preservando comportamento.	**M6:** apresentação final.

Divisão recomendada de responsabilidades

Frente	Responsabilidades
Linguagem	Gramática, tokens, exemplos e especificação.
Frontend	Lexer, parser, AST e mensagens de erro.
Semântica	Tabela de símbolos, escopos, tipos e validações.
Backend	IR, interpretador, geração de Python e runtime.
Qualidade e artigo	Testes, integração, documentação, experimentos e artigo científico.

Critérios de aceite da entrega final

• Compilar ao menos três programas GameLang válidos.

• Rejeitar programas inválidos com mensagens que indiquem linha e coluna.

• Validar nomes, escopos, propriedades e tipos antes da geração de código.

• Garantir equivalência de comportamento entre interpretador e Python gerado.

• Demonstrar ao menos duas otimizações que preservem o comportamento observável.

• Disponibilizar repositório reprodutível, manual de execução, testes e artigo científico.

Fluxo técnico esperado

Programa GameLang → Lexer → Parser → AST → Análise semântica → IR → Otimização → Interpretador ou código Python gerado.

Base de alinhamento: plano de ensino da disciplina Projeto e Implementação de Compiladores, UNASP-SP, semestre 2026/2; especificação inicial da GameLang entregue pela equipe.